



CLAUDE CODE: FROM FIRST INSTALL TO LIFE AFTER PRODUCTION

Context Is The Product

Claude Code deep dive, episode 2

Something in your conversation is being deleted right now, in a fixed order.

Seven things live in your context window and only one of them is what you typed. The rest arrived because of something the agent did on your behalf.

When the window fills, things are removed in a fixed order, and the first thing to go is not the one you would have picked.

Claude Code changes with every release. Every figure, default and feature name on this sheet was checked on the date below and is stated as of that date. Check the current documentation before you rely on any of it.

In scope

- What actually loads into the context window
- The order things are dropped in
- Why "detailed and early" is the profile of what is lost
- Control 1: CLAUDE.md and Compact Instructions
- Control 2: skills, which load on demand
- Control 3: subagents and their own window
- Control 4: MCP tool definitions, deferred

Out of scope

- How to write a skill, which is episode 6
- Choosing MCP servers, which is episode 7
- Orchestrating subagents, which is episode 8
- Token pricing: this is about what earns its place, not what it costs

The junior version of this skill is "use fewer tokens". That is the wrong optimisation and it makes results worse.

01 What loads, and who put it there

Conversation history	you
Every file it has read	the agent
Output of every command	the agent
CLAUDE.md	you, once per session
Auto memory	the agent, first 200 lines or 25KB
Loaded skills	either, on demand
System instructions	the harness

These are interview assumptions chosen to make the behaviour concrete. They are not published figures from any provider and no claim is made about any particular deployment.

03 Only one row is yours

Look at the table above and notice the shape of it. Six of the seven arrived because of something the agent did, not something you wrote.

Which is why rewriting your prompt to be cleverer so often changes nothing. You were editing the one line that was already fine.

Auto memory is the one people do not know about: learnings saved as it works, of which the first 200 lines or 25KB load each session, whichever comes first.

The context window holds conversation history, file contents, command outputs, CLAUDE.md, auto memory, loaded skills, and system instructions.

Claude Code docs, How Claude Code works, checked September 2026

The first 200 lines or 25KB of MEMORY.md, whichever comes first, load at the start of each session.

Claude Code docs, How Claude Code works, checked September 2026

04 The order things are removed in

Older tool outputs are cleared FIRST. Then, if that is not enough, the conversation is summarised.

Requests and key code snippets are preserved. Detailed instructions from early in the conversation may not be.

That last sentence is the whole episode. The most valuable thing you ever type is usually a detailed instruction, given early, which is exactly the profile of what goes.

Claude Code clears older tool outputs first, then summarises the conversation if needed. Requests and key code snippets are preserved; detailed instructions from early in the conversation may be lost.

Claude Code docs, How Claude Code works, checked September 2026

If a single file or tool output is so large that context refills immediately after each summary, Claude Code stops auto-compacting after a few attempts and shows an error rather than looping.

Claude Code docs, How Claude Code works, checked September 2026

05 Control 1: CLAUDE.md, and the section almost nobody uses

A file read at the start of every session is not part of the conversation being summarised, so it cannot be summarised away.

The less known half: you can add a Compact Instructions section to that file saying what should survive a compaction, or run compact with a focus and say what to keep.

Most people wait for compaction to happen. You can decide when, and what matters.

Put persistent rules in CLAUDE.md rather than relying on conversation history.

Claude Code docs, How Claude Code works, checked September 2026

To control what is preserved during compaction, add a "Compact Instructions" section to CLAUDE.md or run /compact with a focus.

Claude Code docs, How Claude Code works, checked September 2026

Bloated CLAUDE.md files cause Claude to ignore your actual instructions, so for each line ask whether removing it would cause a mistake.

Claude Code docs, Best practices, checked September 2026

06 Control 2: skills load on demand

At the start of a session Claude sees only the DESCRIPTIONS. The body of a skill loads only when the skill is used.

So a hundred page reference costs you a one line description until the ten minutes you actually need it.

For skills you invoke manually, disable-model-invocation keeps even the description out of the way.

Unlike CLAUDE.md content, a skill's body loads only when it is used, so long reference material costs almost nothing until you need it.

Claude Code docs, Skills, checked September 2026

Claude sees skill descriptions at session start; setting disable-model-invocation keeps descriptions out of context until you need them.

Claude Code docs, How Claude Code works, checked September 2026



07 Control 3: whose window pays for the work

A subagent works in its own context window. It starts fresh unless it is a fork, its tool calls stay out of your window entirely, and you get back a summary.

Ask the main session to read forty files and forty file contents land in your window and stay there. Ask a subagent and you get the answer instead.

That is the difference between a session still sharp after three hours and one that is confused by lunchtime.

A subagent works in its own context window; its tool calls stay out of your context and Claude gets back a summary when it finishes.

Claude Code docs, How Claude Code works, checked September 2026

A fork starts with a copy of your conversation so far rather than fresh, and its tool calls still stay out of your window.

Claude Code docs, Subagents, checked September 2026

08 Control 4: a cost you did not know you were paying

Each MCP server brings tool definitions with it. Connect six and that could be a great deal of your window spent before you ask anything.

At the time of writing, those definitions are deferred by default and loaded on demand through tool search, so only names and server instructions cost you until a tool is used.

Worth knowing, because when a session feels sluggish the instinct is to blame the model, and the cause may be six servers you connected in March.

MCP tool definitions are deferred by default and loaded on demand via tool search, so only tool names and server instructions consume context until Claude uses a specific tool.

Claude Code docs, How Claude Code works, checked September 2026

09 Why "use fewer tokens" is the wrong optimisation

Writing shorter prompts and being terse makes results worse, because the thing you trimmed was usually the context that would have made the answer right.

The senior version is: decide what earns its place. A precise rule in CLAUDE.md earns it, because it is read once per session and prevents a class of mistake. Forty files of search results do not, so they go to a subagent. A reference earns it only for the ten minutes you need it, so it becomes a skill.

Same window. Completely different contents.

Most best practices follow from one constraint: the context window fills fast and performance degrades as it fills.

Claude Code docs, Best practices, checked September 2026

Trimming context indiscriminately removes the material that would have made the answer correct, which is why brevity is not the same as efficiency.

First principles, not vendor documentation

Questions to ask when quality drops mid-session

1. Run /context: which of the seven is actually eating the window?
2. Is it tool output from one large file read?
3. Has the session compacted since your key instruction?
4. Is that instruction in CLAUDE.md, or only in the conversation?
5. Does CLAUDE.md have a Compact Instructions section?
6. Should this reference be a skill instead of a permanent load?
7. Should this search have gone to a subagent?
8. How many MCP servers are connected, and do you use them all?

Everything else

-  **Site**
<https://code-with-sam-dev.github.io>
-  **YouTube**
<https://www.youtube.com/@CodewithSam-Dev>
-  **LinkedIn**
<https://www.linkedin.com/company/code-with-sam-dev>
-  **x**
<https://x.com/CodeWithSamDev>
-  **TikTok**
<https://www.tiktok.com/@codewithsamdev>
-  **Instagram**
<https://www.instagram.com/codewithsamdev>
-  **Facebook**
<https://www.facebook.com/codewithsamdev>
-  **GitHub**
<https://github.com/code-with-sam-dev>
-  **Buy me a coffee**
<https://buymeacoffee.com/codewithsam>
-  **Buy me sadza**
<https://buymesadza.com/codewithsam>
- The context window**
<https://code.claude.com/docs/en/context-window>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Claude and Claude Code are products of Anthropic. This is an independent, unofficial guide produced by Code with Sam. It is not affiliated with, endorsed by, or sponsored by Anthropic, and no Anthropic artwork is reproduced here. Product names appear only to identify the software being discussed. Other names are the property of their respective owners.

Technical claims verified 2026-09-13 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.